

Use Case 10: Infrastructure as Code

- **Scenario:** Automate infrastructure provisioning.
- **Tasks:**
 - Use CloudFormation to create a stack for a web application.
 - Include EC2, RDS, and an S3 bucket in the stack.
 - Update the stack to add a new feature.

Step 1: Define Requirements

1. The web application stack will include:
 - **EC2 instance:** For hosting the application.
 - **RDS instance:** For the database backend.
 - **S3 bucket:** For static file storage (e.g., images, documents).

2. The stack should be secure, cost-effective, and scalable.
 3. Use **CloudFormation templates** to automate resource provisioning and ensure version control.
-

Step 2: Create a Basic CloudFormation Template

1. **Open CloudFormation Console:**
 - Navigate to the **AWS CloudFormation Console**.
 2. **Write the Template:**
 - Use **YAML** or **JSON** to define your infrastructure.
 3. **Save the Template:**
 - Save the file as `web-app-stack.yaml`.
-

Step 3: Deploy the CloudFormation Stack

1. **Upload the Template:**

- In the CloudFormation Console, click **Create Stack** and choose **With New Resources**.
 - Upload the web-app-stack.yaml file.
 - 2. **Specify Parameters:**
 - Enter required parameters like EC2 instance type and key pair.
 - 3. **Review and Launch:**
 - Verify the resources and permissions.
 - Click **Create Stack**.
 - 4. **Monitor the Stack Creation:**
 - Wait for the stack status to reach **CREATE_COMPLETE**.
-

Step 4: Secure and Optimize the Stack

1. **Secure the EC2 Instance:**
 - Restrict **SSH access** to specific IPs in the WebAppSecurityGroup.

- Enable **AWS Systems Manager Session Manager** for secure remote access without SSH:
 - Attach the AmazonSSMManagedInstanceCore policy to the EC2 instance's IAM role.

2. **Secure the RDS Instance:**

- Use **KMS encryption** for the RDS storage.
- Ensure the RDS instance is in a **private subnet** for better security.

3. **Secure the S3 Bucket:**

- Block public access by enabling **Block Public Access Settings**.

1. **Cost Optimization:**

- Use t3.micro instances for EC2 and RDS to stay within the **AWS Free Tier** (if applicable).

- Enable **Auto Scaling** for the EC2 instance to handle variable loads efficiently.
 - Set up **lifecycle policies** to transition old S3 objects to **Glacier**.
-

Step 5: Update the Stack to Add a New Feature

1. **New Feature:**
 - Add an **Elastic Load Balancer (ELB)** to distribute traffic to multiple EC2 instances.
3. **Update the Stack:**
 - In the CloudFormation Console, click on your stack and choose **Update**.
 - Upload the modified template and deploy the update.
4. **Validate the Update:**

- Test the ELB endpoint to confirm traffic is properly distributed to EC2 instances.
-

Step 6: Monitor and Maintain the Stack

1. **Enable Logging:**

- Use **CloudTrail** and **CloudWatch Logs** to track access and monitor resource usage.
- Set up alarms for unusual activity.

2. **Automate Backups:**

- Enable automated backups for RDS with a defined retention period.
- Use **S3 Versioning** to preserve previous versions of objects.

3. **Optimize Costs:**

- Use **Reserved Instances** or **Savings Plans** for predictable workloads.

- Periodically review and delete unused resources.